Your Agents

A Crewed-Agent AI Chat Platform

MCP Tools
Knowledge Base
Agent Observability
Privacy by Design

Agenda

1. Overview
2. Features
3. Architecture
4. Privacy & Security

Overview

What is Your Agents?

A privacy-first AI chat platform

Users bring their own API keys and converse with LLMs through **OpenRouter** or **OpenAI**. Every request is enriched by a three-layer pipeline: knowledge retrieval, MCP tool execution, and streaming LLM completion — all orchestrated client-side.

Framework	Next.js 15 App Router
UI	React 19, Tailwind CSS 4
Language	TypeScript
Storage	IndexedDB + LocalStorage
LLM API	OpenRouter / OpenAI
Tool Protocol	MCP over HTTP

Design Goals

- **Client-first privacy** — API keys never leave the browser
- **Extensible agent configuration** — system prompts, models, and tools per agent
- **Observable agentic execution** — live reasoning, task list, and tool call results
- **Persistent sessions** — full chat history stored locally in IndexedDB
- **Pluggable MCP tool ecosystem** — SQL, LMS, Image, Browser connectors

Features

Features

Agent Management

Create agents with distinct system prompts, model choices, and tool configs.

Import / export via JSON for easy sharing.

Knowledge Management

Per-agent knowledge store with custom generation prompts. Relevant entries are semantically retrieved and injected on every request.

Session Management

Named chat sessions in IndexedDB. Auto-generated titles via LLM. Full message history with per-message cost tracking.

MCP Tool Execution

Built-in connectors:

- **MySQL** — read-only SQL
- **LMS** — Canvas, Brightspace, Blackboard
- **Image** — gen & processing
- **Browser** — scraping & automation

Observability

Live sidebar shows:

- Planner reasoning
- Task list
- Tool calls + raw results

Historical runs replayed from saved snapshots.

PWA Support

Installable as a native-like app on any device. Supports web push notifications via VAPID and works offline for cached sessions.

Privacy Features

User-provided API Keys

No vendor lock-in. Users supply their own OpenRouter or OpenAI key. Keys are stored exclusively in `localStorage` — the server never sees them.

Client-side API Calls

All LLM requests travel **directly** from the browser to OpenRouter. No proxying through the application server — zero server-side logging of conversation content.

Local Data Storage

Chat history and agent configurations live in **IndexedDB** inside the user's browser. Nothing is synced to a backend database.

No Server Processing

The Next.js server only serves static assets and MCP tool endpoints. It does not process, store, or log any user messages or API responses.

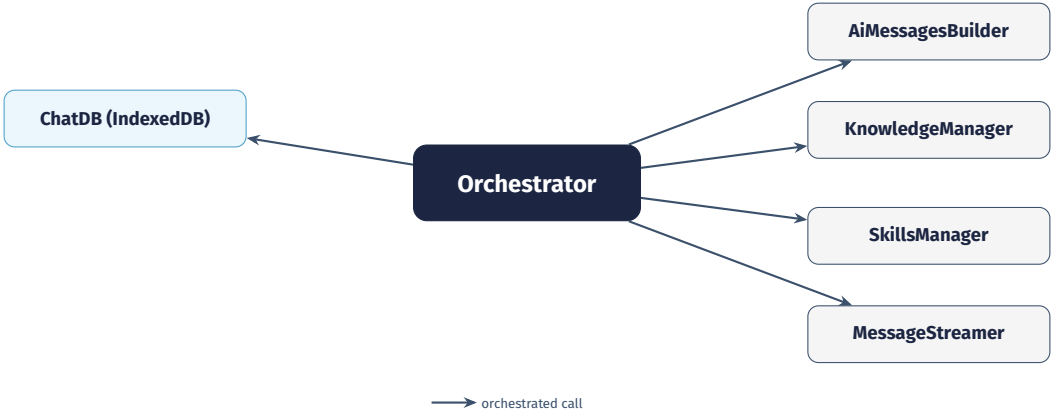
Secured Tools

MCP tool endpoints enforce strict safety guarantees:

- MySQL access is **read-only** only
- Sensitive fields are **masked** in results
- Row count capped at **20** by default
- Credentials never appear in API responses

Architecture

Core Components



Message Flow

1. User input — text or image (base64-encoded for vision models)

↓

2. AiMessagesBuilder — sliding window of last 10 msgs + agent system prompt

↓

3. KnowledgeManager — semantic retrieval, inject as system message [if configured]

↓

4. SkillsManager — plan tasks, execute MCP tools, inject results [if enabled]

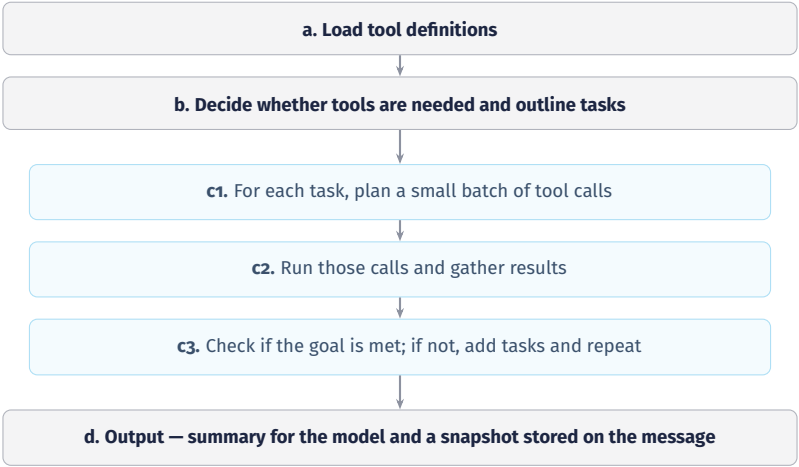
↓

5. MessageStreamer — streaming LLM call, token-by-token UI update

↓

6. ChatDB — persist all messages + AgentRunSnapshot

SkillsManager — Internal Pipeline



Privacy & Security

Privacy & Security

Client-side guarantees

- API keys stored only in `localStorage`
- Chat history & Agent configs lives in IndexedDB, never sent to the server
- LLM calls **directly** from browser → OpenRouter

Server-side guarantees

- MySQL MCP is **read-only** — only `SELECT`, `SHOW`, `DESCRIBE`
- Sensitive fields masked in all query results
- `LIMIT` enforced on each query (20 rows)

Storage model

Data	Location	Accessible by
API keys	localStorage	User only
Chat history	IndexedDB	User only
Agent configs	IndexedDB	User only

Thank You

Questions & Discussion